AI Loan Adjudication: Final Report

Sabber Ahamed September 3, 2026

Business problem

I treated this as a product and risk problem, not only a modeling exercise. The system must understand an applicant's conversation, apply the current lending rules consistently, and explain the result without inventing a reason or citing the wrong rule.

The business opportunity is a lower-cost workflow that can automate clear cases and send uncertain cases to a person. Keeping client policy outside the model also makes new-client onboarding and rule updates possible without training a separate model for every ruleset.

System I built

I used a small fine-tuned language model with a deterministic rules engine:

```
Customer dialogue
      ↓
Qwen3-1.7B + LoRA
Extract 10 fields and produce a shadow result
      ↓
Schema and evidence validation
      ↓
Deterministic rules engine
Recalculate the decision and failed rule IDs
      ↓
Verified customer explanation and audit record
```

- **Language model:** interprets informal, corrected, incomplete, and out-of-order dialogue. It returns ten application fields, a shadow decision, failed rule IDs, and a short explanation in JSON.
- **Rules engine:** evaluates every active JSON rule again. Rejection takes priority over human review, and human review takes priority over approval. Missing information blocks a lending decision.
- **Explanation layer:** creates the customer response from the verified decision and rule results. The model's free-form explanation is not authoritative.

Why I kept the rules deterministic

Lending explanations carry a higher cost of error than ordinary chatbot responses. The U.S. Consumer Financial Protection Bureau states that adverse-action reasons must be specific and accurately describe the factors considered. A complex algorithm does not remove that obligation. CFPB Circular 2022-03

I therefore use the model for language understanding and code for decision authority. This gives the system:

- Reproducible decisions for the same application and policy version.
- Verified rule citations, observed values, and thresholds.
- Policy updates without model retraining.
- A record that operations, compliance, and customer-support teams can inspect.
- A clear trigger for human review when the model and rules engine disagree.

This layer does not prove fair-lending compliance. Policy design, data use, protected-class handling, and outcomes still require legal, compliance, and fairness review.

Why I selected Qwen3-1.7B

I chose Qwen3-1.7B because this is a narrow structured-output task. The model does not need to memorize lending policy because the rules arrive with each request and the rules engine evaluates them independently.

- A 1.7B model needs less memory and compute than a 14B serving model, which should reduce inference cost and latency. Production benchmarks are still required.
- A larger model may interpret difficult language better, but it still cannot guarantee correct thresholds or citations.
- LoRA produced an adapter of about 84 MB, making it easier to store, version, and replace.
- Rule changes remain in configuration rather than a policy-specific adapter.

Data and training

The supplied material contained rules but no application conversations, so I created the dataset first.

- **Core data:** 4,000 training, 500 validation, and 500 clean test records.
- **Test-2:** 500 adversarial and messy conversations excluded from training.
- **Test-3:** 500 conversations evaluated with unseen rule changes and no retraining.
- **Coverage:** complete applications, missing values, boundaries, categorical aliases, corrections, contradictions, ambiguity, irrelevant text, and typos.
- **Ground truth:** code calculated canonical values, decisions, failed rule IDs, and explanations.
- **Training:** QLoRA for two epochs and 250 steps.

I published the model adapter and 6,000 dataset records on Hugging Face.

Evaluation results

I measured the model in generation mode and reported the model and deterministic layers separately.

Evaluation set	Purpose	Model decision	Rules-engine decision	Model citations	Verified citations
Validation	Model selection	99.4%	99.4%	97.2%	100.0%
Test-1	Clean unseen cases	99.4%	100.0%	95.8%	100.0%
Test-2	Adversarial dialogue	87.8%	88.8%	91.8%	95.6%
Test-3	Changed rules	78.6%	91.6%	53.0%	75.4%

Key findings:

- **Clean test:** the rules engine corrected the remaining decision and citation errors.
- **Adversarial test:** the rules engine corrected 6 decisions and 20 citation sets when field extraction was accurate.
- **Changed-rules test:** deterministic recomputation corrected 94 decisions and 129 citation sets. Expected-rejection false approvals fell from 19 to zero.
- **Remaining weakness:** vague or contradictory dialogue sometimes became a confident but incorrect field value. The rules engine cannot repair a wrong input that appears valid.

Business conclusion and production boundary

The experiment supports a hybrid product. A small model can keep the language layer relatively inexpensive, while deterministic execution avoids the cost and latency of a second verification model. Configurable rules can shorten policy updates and support new clients without retraining. Verified outcomes may also reduce routine manual checking while giving current clients an explanation they can audit.

I would not deploy this prototype for unsupervised real-world credit decisions yet. The results measure fidelity to synthetic conversations and supplied rules, not borrower risk or regulatory approval. Before a production pilot, I would add:

- Evidence grounding for every extracted value.
- Stronger contradiction and ambiguity detection.
- Versioned audit logs and access controls.
- Evaluation on representative real-world data.
- Latency and inference-cost benchmarks.
- Human review for missing information, conflicting evidence, configured review outcomes, and model-engine disagreement.